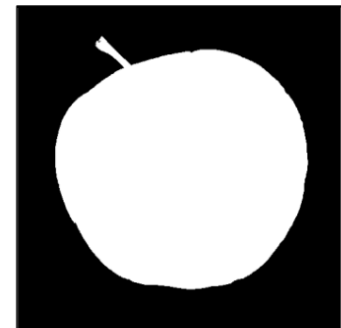
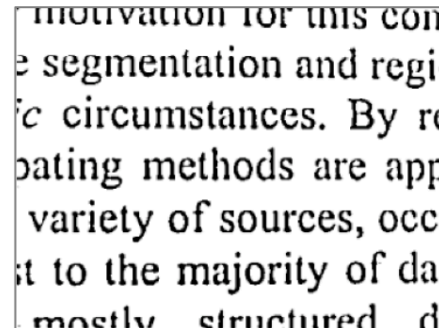
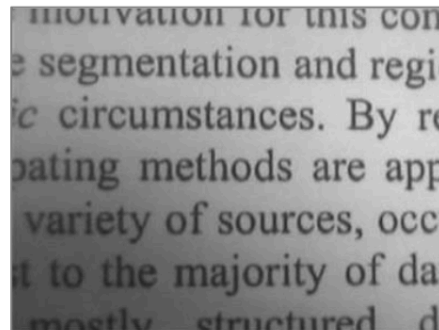
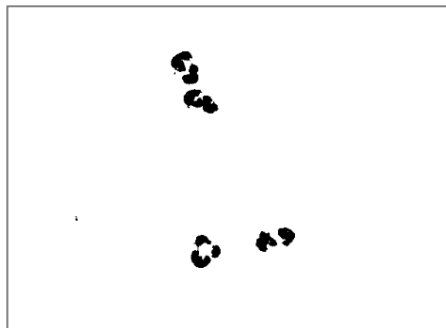
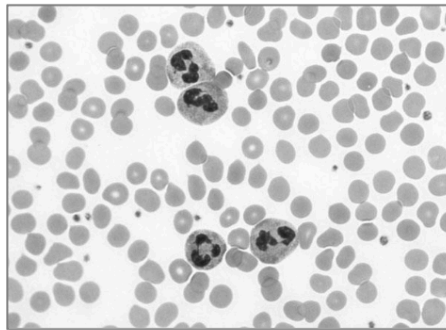


이진 영상 처리

1) 영상의 이진화

영상의 이진화

- 영상의 이진화(Binarization)란?
 - 영상의 픽셀 값을 0 또는 255(1)로 만드는 연산
 - 배경(background) vs. 객체(object)
 - 관심 영역 vs. 비관심 영역



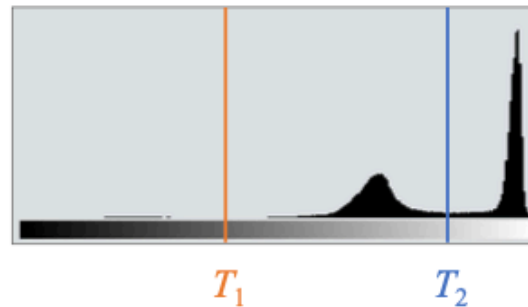
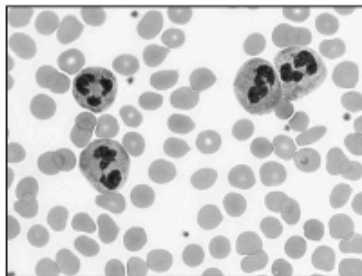
영상의 이진화

■ 그레이스케일 영상의 이진화

$$g(x, y) = \begin{cases} 0 & \text{if } f(x, y) \leq T \\ 255 & \text{if } f(x, y) > T \end{cases}$$



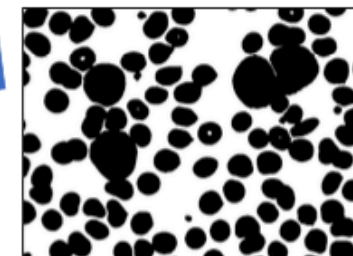
■ T : 임계값, 문턱치, threshold



히스토그램



T1



T2

영상의 이진화

■ 임계값 함수

```
cv2.threshold(src, thresh, maxval, type, dst=None) -> retval, dst
```

- src: 입력 영상. 다채널, 8비트 또는 32비트 실수형.
- thresh: 사용자 지정 임계값
- maxval: cv2.THRESH_BINARY 또는 cv2.THRESH_BINARY_INV 방법 사용 시 최댓값. 보통 255로 지정.
- type: cv2.THRESH_로 시작하는 플래그.
임계값 함수 동작 지정 또는 자동 임계값 결정 방법 지정
- retval: 사용된 임계값
- dst: 출력 영상. src와 동일 크기, 동일 타입, 같은 채널 수.

영상의 이진화

■ cv2.threshold() 함수 동작 타입



cv2.THRESH_BINARY

$$\text{dst}(x, y) = \begin{cases} \text{maxval} & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$



cv2.THRESH_BINARY_INV

$$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{maxval} & \text{otherwise} \end{cases}$$

cv2.THRESH_TRUNC

$$\text{dst}(x, y) = \begin{cases} \text{threshold} & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$$

cv2.THRESH_TOZERO

$$\text{dst}(x, y) = \begin{cases} \text{src}(x, y) & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$

cv2.THRESH_TOZERO_INV

$$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$$

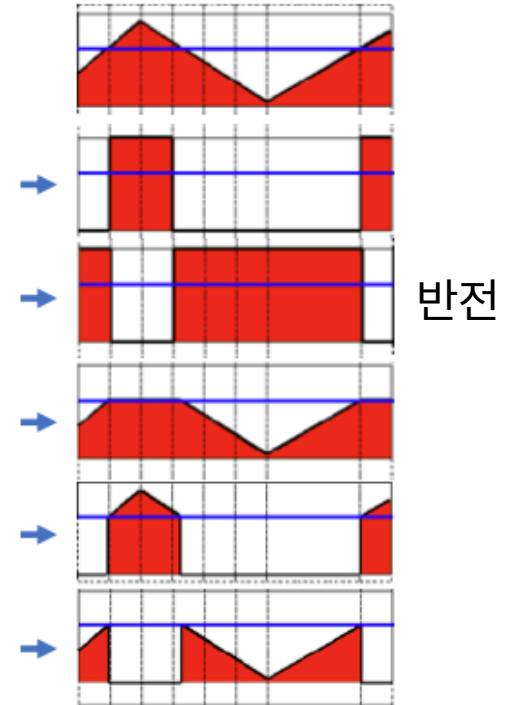


cv2.THRESH_OTSU

Otsu 알고리즘으로 임계값 자동 결정

cv2.THRESH_TRIANGLE

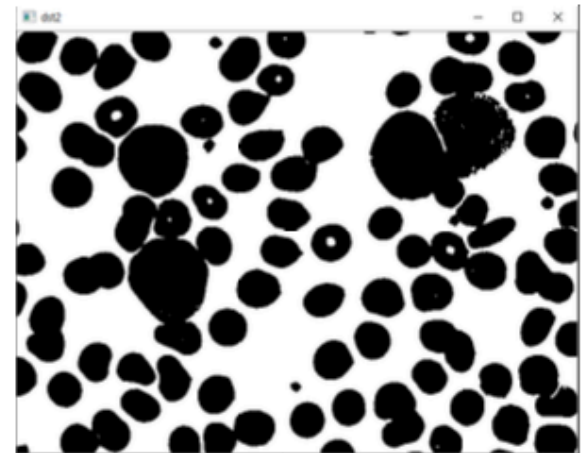
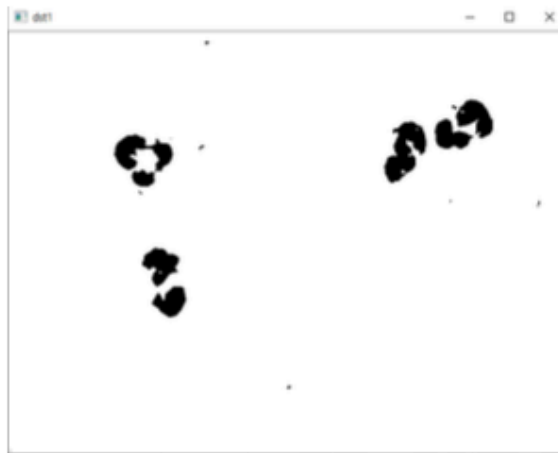
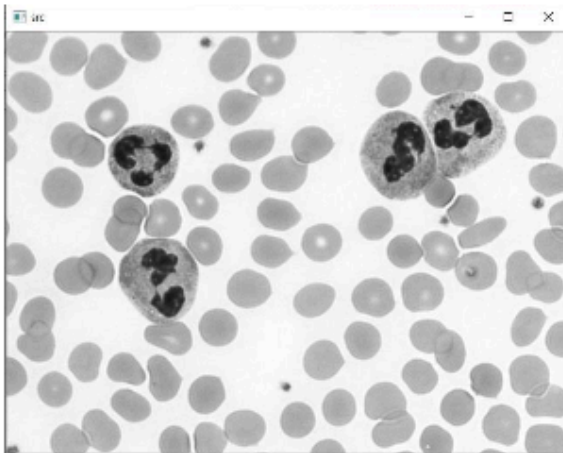
삼각 알고리즘으로 임계값 자동 결정



영상의 이진화

■ 영상의 이진화 예제

```
src = cv2.imread('cells.png', cv2.IMREAD_GRAYSCALE)  
  
_, dst1 = cv2.threshold(src, 100, 255, cv2.THRESH_BINARY)  
_, dst2 = cv2.threshold(src, 210, 255, cv2.THRESH_BINARY)
```



염색이 되어 있는만 출력됨

영상의 이진화

■ 트랙바를 이용한 영상의 이진화 예제

```
src = cv2.imread('cells.png', cv2.IMREAD_GRAYSCALE)

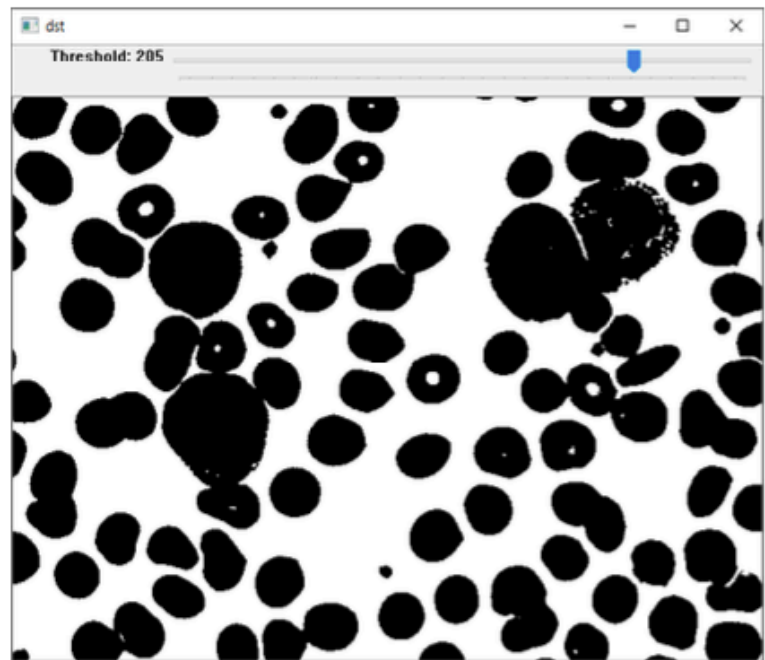
def on_threshold(pos):
    _, dst = cv2.threshold(src, pos, 255, cv2.THRESH_BINARY)
    cv2.imshow('dst', dst)

cv2.imshow('src', src)
cv2.namedWindow('dst')
cv2.createTrackbar('Threshold', 'dst', 0, 255, on_threshold)
cv2.setTrackbarPos('Threshold', 'dst', 128)

cv2.waitKey()
cv2.destroyAllWindows()
```

영상의 이진화

- 트랙바를 이용한 영상의 이진화 예제 실행 결과



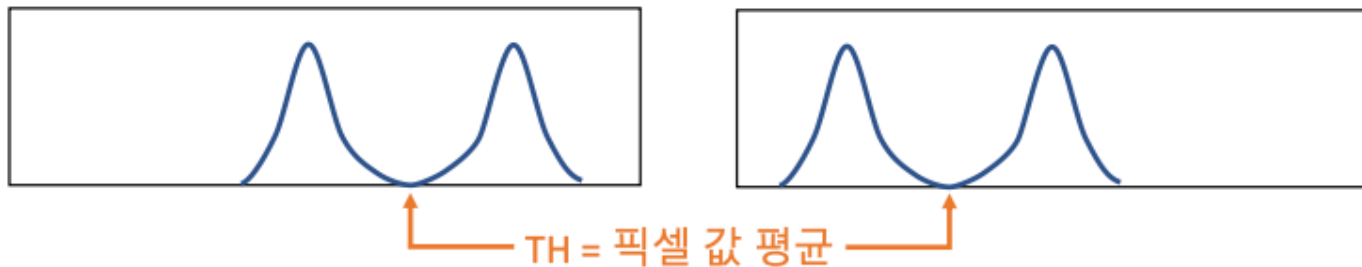
이진 영상 처리

2) 자동 이진화 : Otsu 방법

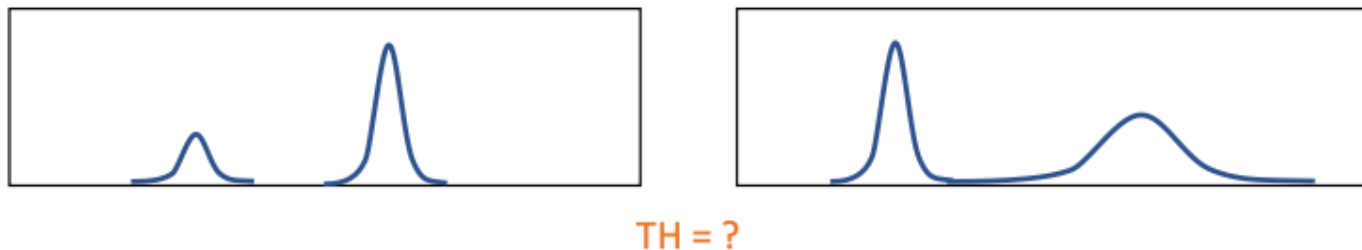
자동 이진화

■ 임계값 자동 결정 방법

- 영상의 히스토그램이 bimodal이고, 전경&배경 픽셀 분포가 비슷하다면?



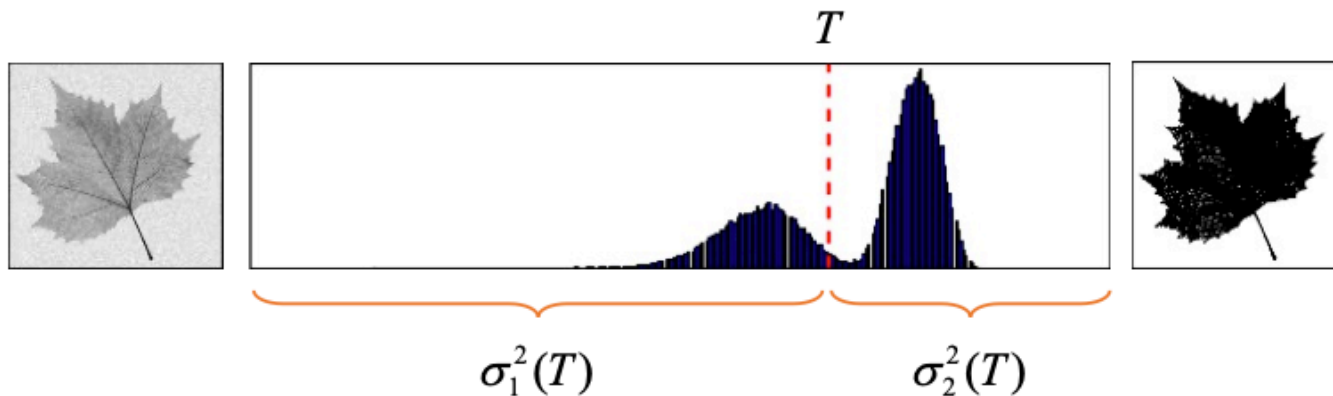
- 히스토그램이 bimodal이지만, 전경&배경 픽셀 분포가 크게 다르다면?



자동 이진화 : Otsu 방법

■ Otsu 이진화 방법

- 입력 영상이 배경(background)과 객체(object) 두 개로 구성되어 있다고 가정
→ Bimodal histogram
- 임의의 임계값 T 에 의해 나뉘지는 두 픽셀 분포 그룹의 분산이 최소가 되는 T 를 선택
- 일종의 최적화 알고리즘(optimization algorithm)



자동 이진화 : Otsu 방법

■ Otsu 이진화 방법

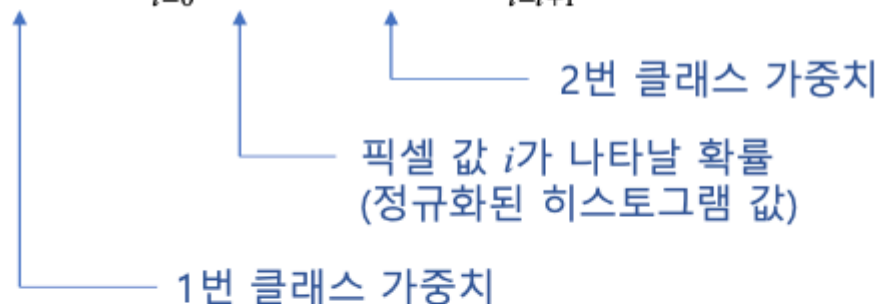
분산

분산

- Within-class variance:

$$\sigma_{Within}^2(t) = \omega_1(t)\sigma_1^2(t) + \omega_2(t)\sigma_2^2(t)$$

$$\omega_1(t) = \sum_{i=0}^t p(i) \quad , \quad \omega_2(t) = \sum_{i=t+1}^{L-1} p(i)$$



- Otsu 임계값:

$$T = \operatorname{argmin}_{t \in \{0,1,\dots,L-1\}} \sigma_{Within}^2(t)$$

자동 이진화 : Otsu 방법

▪ Otsu 이진화 방법

- Within-class variance 최소화 → Between-class variance 최대화

$$\begin{aligned}\sigma_{Between}^2(t) &= \sigma^2 - \sigma_{Within}^2(t) \\ &= \omega_1(t)(1 - \omega_1(t))(\mu_1(t) - \mu_2(t))^2\end{aligned}$$

- 모든 t 값에 대해 $\sigma_{Between}^2(t)$ 를 구하여 최적의 T 를 선택 → Slow
- Recursion을 이용한 효율적 계산 → Fast

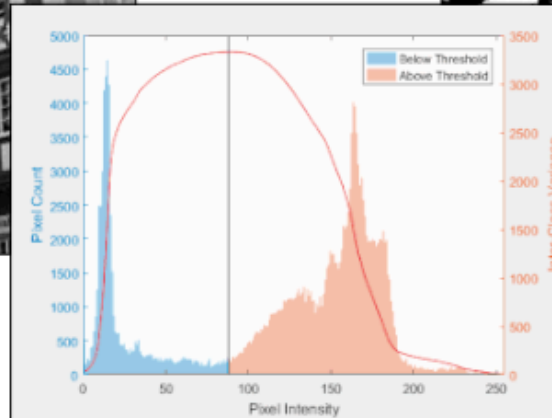
$$\omega_1(0) = p(0), \mu_1(0) = 0$$

$$\omega_1(t) = \omega_1(t-1) + p(t)$$

$$\mu_1(t) = \frac{\omega_1(t-1)\mu_1(t-1) + tp(t)}{\omega_1(t)} \quad \mu_2(t) = \frac{\mu - \omega_1(t)\mu_1(t)}{1 - \omega_1(t)}$$

자동 이진화 : Otsu 방법

■ Otsu 이진화 방법



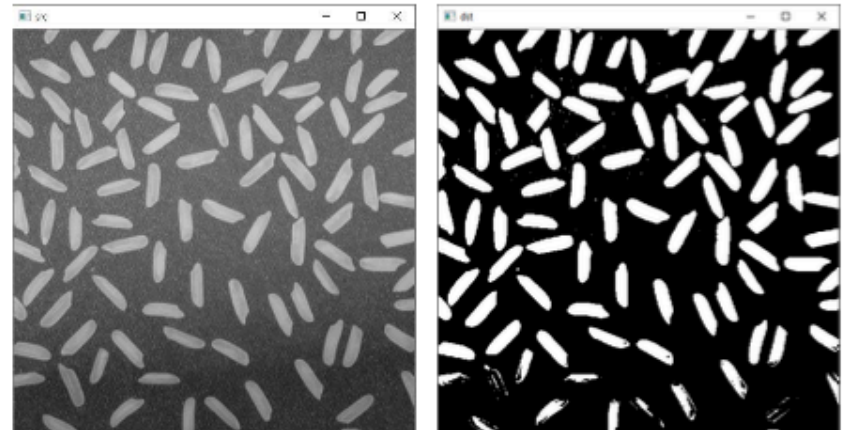
자동 이진화 : Otsu 방법

- Otsu 방법을 이용한 자동 이진화

```
src = cv2.imread('rice.png', cv2.IMREAD_GRAYSCALE)

th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)
print("otsu's threshold:", th) # 131
```

```
관리자: C:\Windows\System32\cmd.exe - python otsu.py
c:\coding\python\opencv\week4>python otsu.py
otsu's threshold: 131.0
```



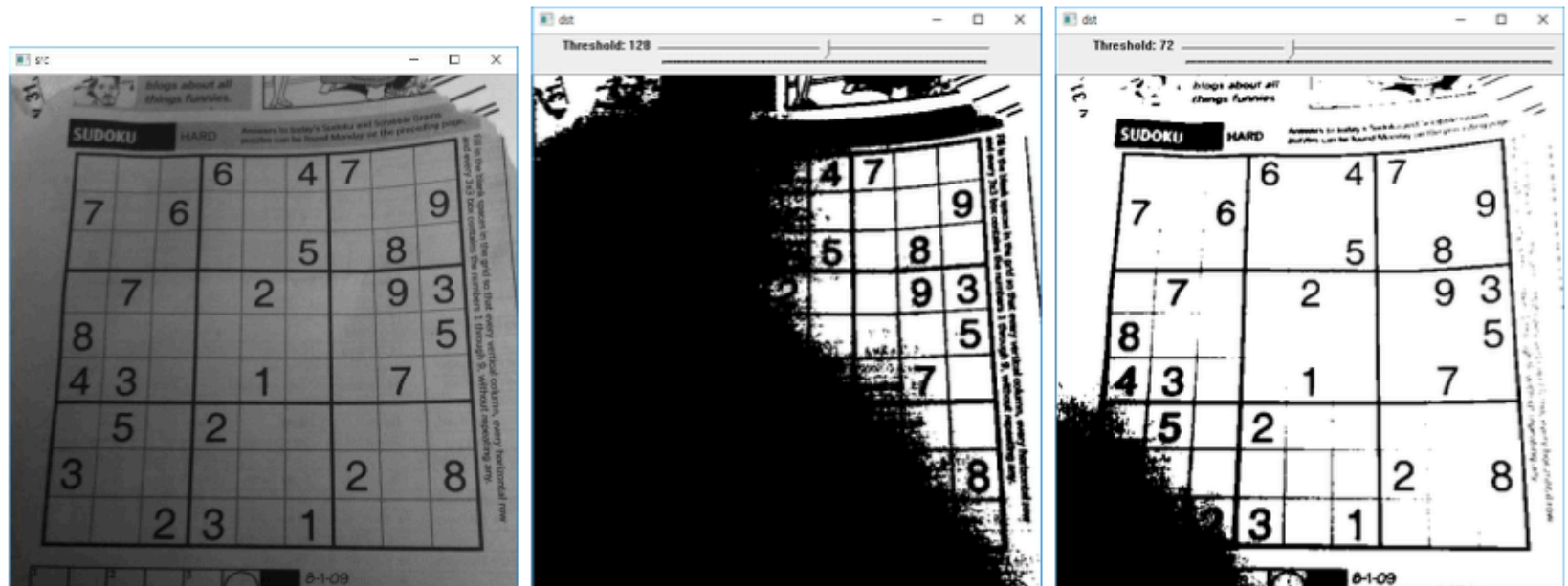
cv2.THRESH_BINARY_INV

이진 영상 처리

3) 지역 이진화

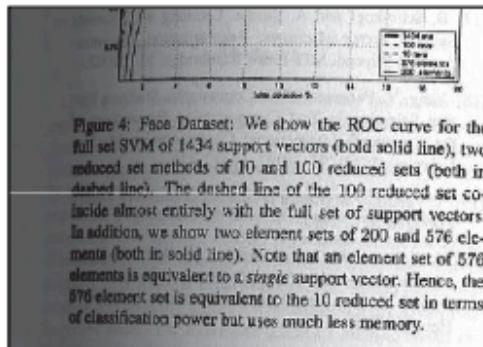
지역 이진화

- 균일하지 않은 조명 환경에서 촬영된 영상의 이진화
 - threshold2.py 프로그램에 sudoku.jpg 파일을 입력으로 사용

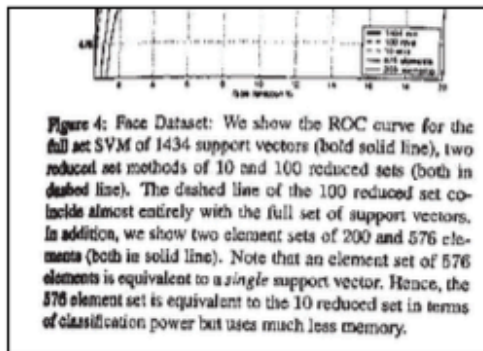
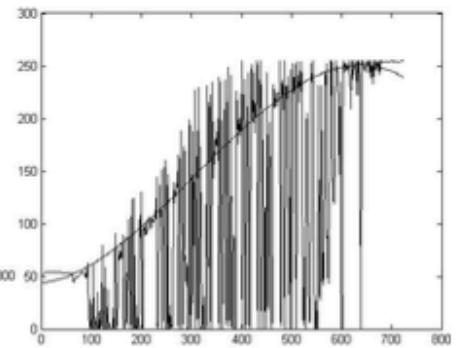
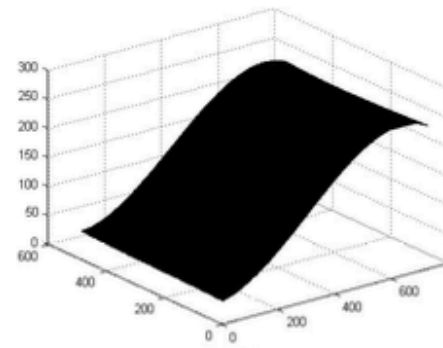


지역 이진화

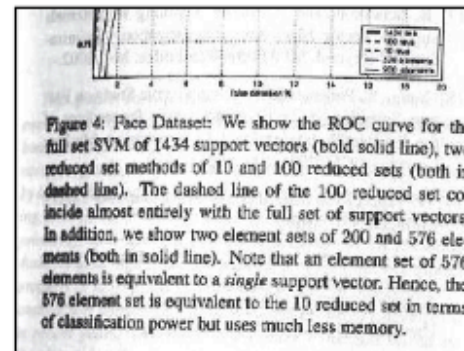
- 균일하지 않은 조명의 영향을 해결하려면?
 - 불균일한 조명 성분을 보상한 후 전역 이진화 수행



Surface Fitting



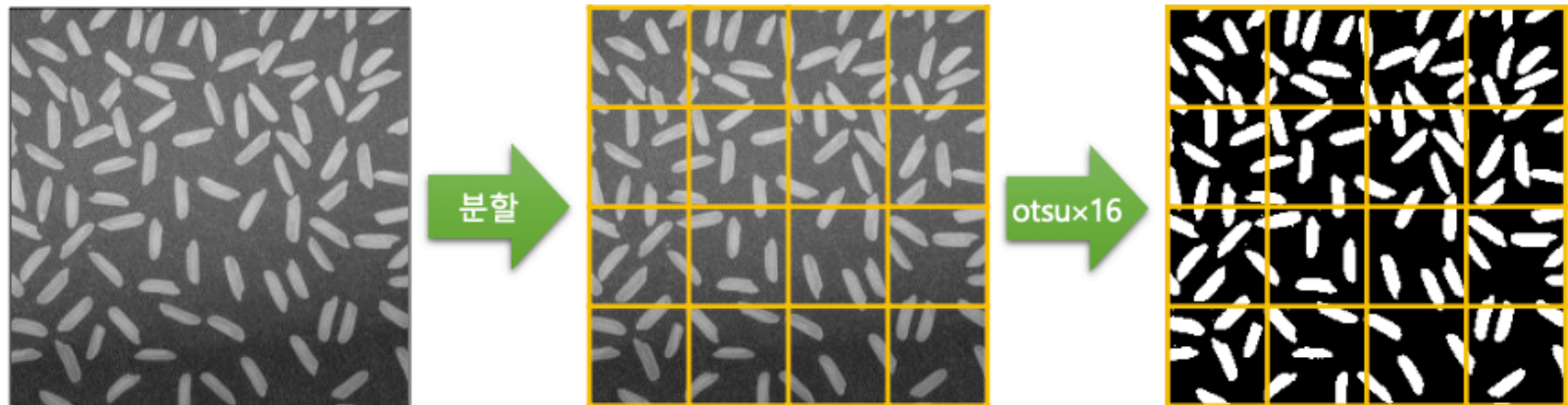
Global Thresholding



Shading Compensation

지역 이진화

- 균일하지 않은 조명의 영향을 해결하려면?
 - 픽셀 주변에 작은 윈도우를 설정하여 지역 이진화 수행
 - 윈도우의 크기는?
 - 윈도우 형태는? Uniform? Gaussian?
 - 윈도우를 겹칠 것인가? Overlap? Non-overlap?
 - 윈도우 안에 배경 또는 객체만 존재한다면?



지역 이진화

■ 지역 이진화 예제

```
src = cv2.imread('rice.png', cv2.IMREAD_GRAYSCALE)

# 전역 이진화 by Otsu's method
_, dst1 = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

# 지역 이진화 by Otsu's method
dst2 = np.zeros(src.shape, np.uint8)

bw = src.shape[1] // 4
bh = src.shape[0] // 4

for y in range(4):
    for x in range(4):
        src_ = src[y*bh:(y+1)*bh, x*bw:(x+1)*bw]
        dst_ = dst2[y*bh:(y+1)*bh, x*bw:(x+1)*bw]
        cv2.threshold(src_, 0, 255, cv2.THRESH_BINARY|cv2.THRESH_OTSU, dst_)
```

지역 이진화

■ 지역 이진화 예제 실행 결과



지역 이진화

■ OpenCV 적응형 이진화

```
cv2.adaptiveThreshold(src, maxValue, adaptiveMethod,  
                     thresholdType, blockSize, C, dst=None) -> dst
```

- src: 입력 영상. 그레이스케일 영상.
- maxValue: 임계값 함수 최대값. 보통 255.
- adaptiveMethod: 블록 평균 계산 방법 지정. cv2.ADAPTIVE_THRESH_MEAN_C는 산술 평균, cv2.ADAPTIVE_THRESH_GAUSSIAN_C는 가우시안 가중치 평균
- thresholdType: cv2.THRESH_BINARY 또는 cv2.THRESH_BINARY_INV 지정
- blockSize: 블록 크기. 3 이상의 홀수.
- C: 블록 내 평균값 또는 블록 내 가중 평균값에서 뺀 값.
(x, y) 픽셀의 임계값으로 $T(x, y) = \mu_B(x, y) - C$ 를 사용

지역 이진화

■ OpenCV 적응형 이진화 예제

```
src = cv2.imread('sudoku.jpg', cv2.IMREAD_GRAYSCALE)

def on_trackbar(pos):
    bsize = pos
    if bsize % 2 == 0:
        bsize = bsize - 1
    if bsize < 3:
        bsize = 3

    dst = cv2.adaptiveThreshold(src, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, bsize, 5)

    cv2.imshow('dst', dst)

cv2.imshow('src', src)
cv2.namedWindow('dst')
cv2.createTrackbar('Block Size', 'dst', 0, 200, on_trackbar)
cv2.setTrackbarPos('Block Size', 'dst', 11)
```


지역 이진화

- OpenCV 적응형 이진화 예제 실행 결과



이진 영상 처리

4) 모폴로지

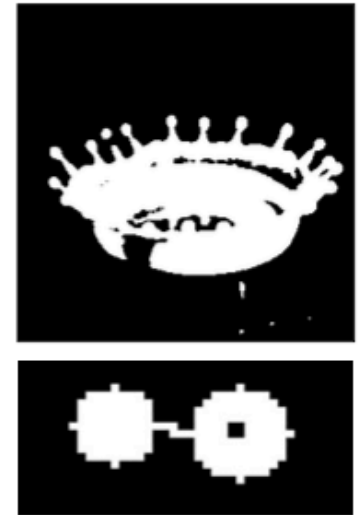
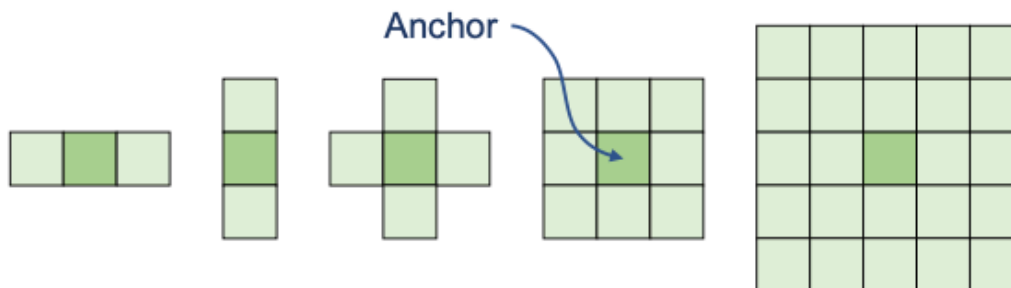
모폴로지

■ 모폴로지(Morphology) 연산이란?

- 영상을 형태학적인 측면에서 다루는 기법
- 다양한 영상 처리 시스템에서 전처리(pre-processing) 또는 후처리(post-processing) 형태로 널리 사용
- 수학적 모폴로지(mathematical morphology)

■ 구조 요소(Structuring element)

- 모폴로지 연산의 결과를 결정하는 커널, 마스크, 윈도우



모폴로지 (1) : 침식과 팽창

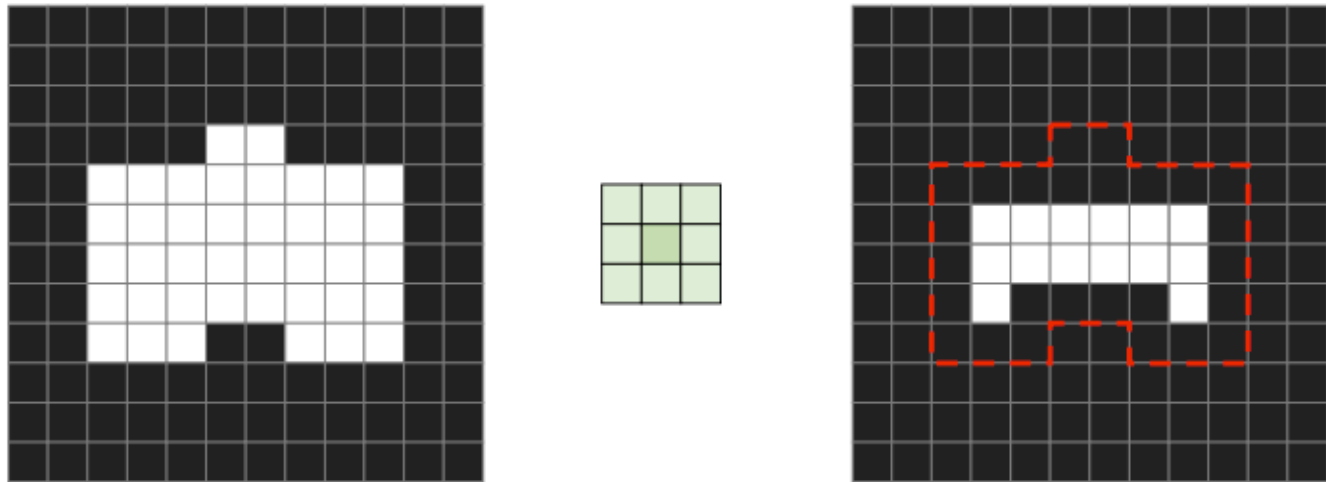
■ 이진 영상의 침식(erosion) 연산

- 구조 요소가 객체 영역 내부에 완전히 포함될 경우 고정점 픽셀을 255로 설정



침식 연산은 객체 외곽을 깎아내는 연산 →

객체 크기는 감소 & 배경은 확대



객체 크기는 확대 & 배경은 축소

모폴로지 (1) : 침식과 팽창

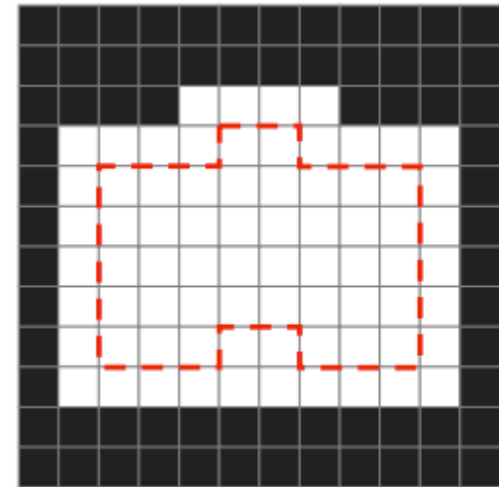
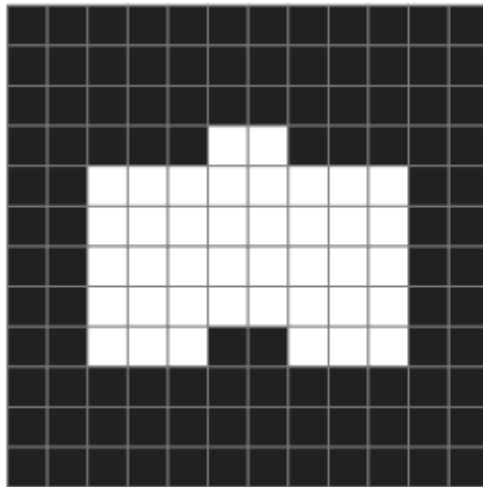
■ 이진 영상의 팽창(dilation) 연산

- 구조 요소와 객체 영역이 한 픽셀이라도 만날 경우 고정점 픽셀을 255로 설정



팽창 연산은 객체 외곽을 확대시키는 연산 →

객체 크기는 확대 & 배경은 축소



모폴로지 (1) : 침식과 팽창

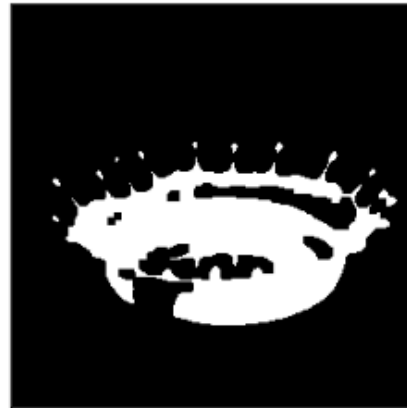
- 실제 영상의 침식 연산 결과
 - 객체 영역(흰색)이 점점 줄어들음
 - 작은 크기의 객체(잡음) 제거 효과



입력 영상



침식 1회



침식 2회



침식 3회

모폴로지 (1) : 침식과 팽창

- 실제 영상의 팽창 연산 결과
 - 객체 영역(흰색)이 점점 불어남
 - 객체 내부의 홀(구멍)이 채워짐



입력 영상



팽창 1회



팽창 2회



팽창 3회

모폴로지 (1) : 침식과 팽창

■ 모폴로지 침식 연산

```
cv2.erode(src, kernel, dst=None, anchor=None, iterations=None,  
          borderType=None, borderValue=None) -> dst
```

- src: 입력 영상.
- kernel: 구조 요소. getStructuringElement() 함수에 의해 생성 가능. 만약 **None**을 지정하면 3x3 사각형 구성 요소를 사용.
- dst: 출력 영상. src와 동일한 크기와 타입
- anchor: 고정점 위치. 기본값 (-1, -1)을 사용하면 중앙점을 사용.
- iterations: 반복 횟수. 기본값은 1.
- borderType: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT인 경우, 확장된 가장자리 픽셀을 채울 값.

모폴로지 (1) : 침식과 팽창

■ 모폴로지 팽창 연산

```
cv2.dilate(src, kernel, dst=None, anchor=None, iterations=None,  
           borderType=None, borderValue=None) -> dst
```

- src: 입력 영상.
- kernel: 구조 요소. `getStructuringElement()` 함수에 의해 생성 가능.
만약 `None`을 지정하면 3x3 사각형 구성 요소를 사용.
- dst: 출력 영상. src와 동일한 크기와 타입
- anchor: 고정점 위치. 기본값 (-1, -1)을 사용하면 중앙점을 사용.
- iterations: 반복 횟수. 기본값은 1.
- borderType: 가장자리 픽셀 확장 방식. 기본값은 `cv2.BORDER_CONSTANT`.
- borderValue: `cv2.BORDER_CONSTANT`인 경우, 확장된 가장자리 픽셀을 채울 값.

모폴로지 (1) : 침식과 팽창

■ 모폴로지 구조 요소 (커널) 생성

```
cv2.getStructuringElement(shape, ksize, anchor=None) -> retval
```

- shape: 구조 요소 모양을 나타내는 플래그

cv2.MORPH_RECT	사각형 모양
cv2.MORPH_CROSS	십자가 모양
cv2.MORPH_ELLIPSE	사각형에 내접하는 타원

- ksize: 구조 요소 크기. (width, height) 튜플.
- anchor: MORPH_CROSS 모양의 구조 요소에서 고정점 좌표.
(-1, -1)을 지정하면 구조 요소의 중앙을 고정점으로 사용.
- retval: 0과 1로 구성된 cv2.CV_8UC1 타입 행렬. [numpy.ndarray](#).
(1의 위치가 구조 요소 모양을 결정.)

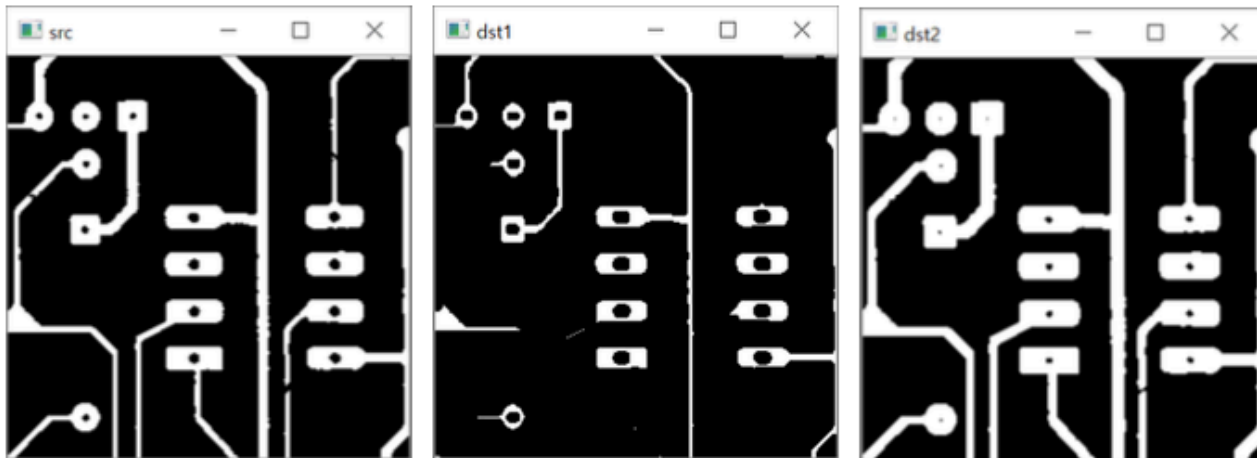
모폴로지 (1) : 침식과 팽창

■ 이진 영상의 침식과 팽창 예제

```
src = cv2.imread('circuit.bmp', cv2.IMREAD_GRAYSCALE)

se = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 3))
dst1 = cv2.erode(src, se)

dst2 = cv2.dilate(src, None)
```



침식

팽창

모폴로지 (2) : 열기와 닫기

- 이진 영상의 열기(opening) 연산

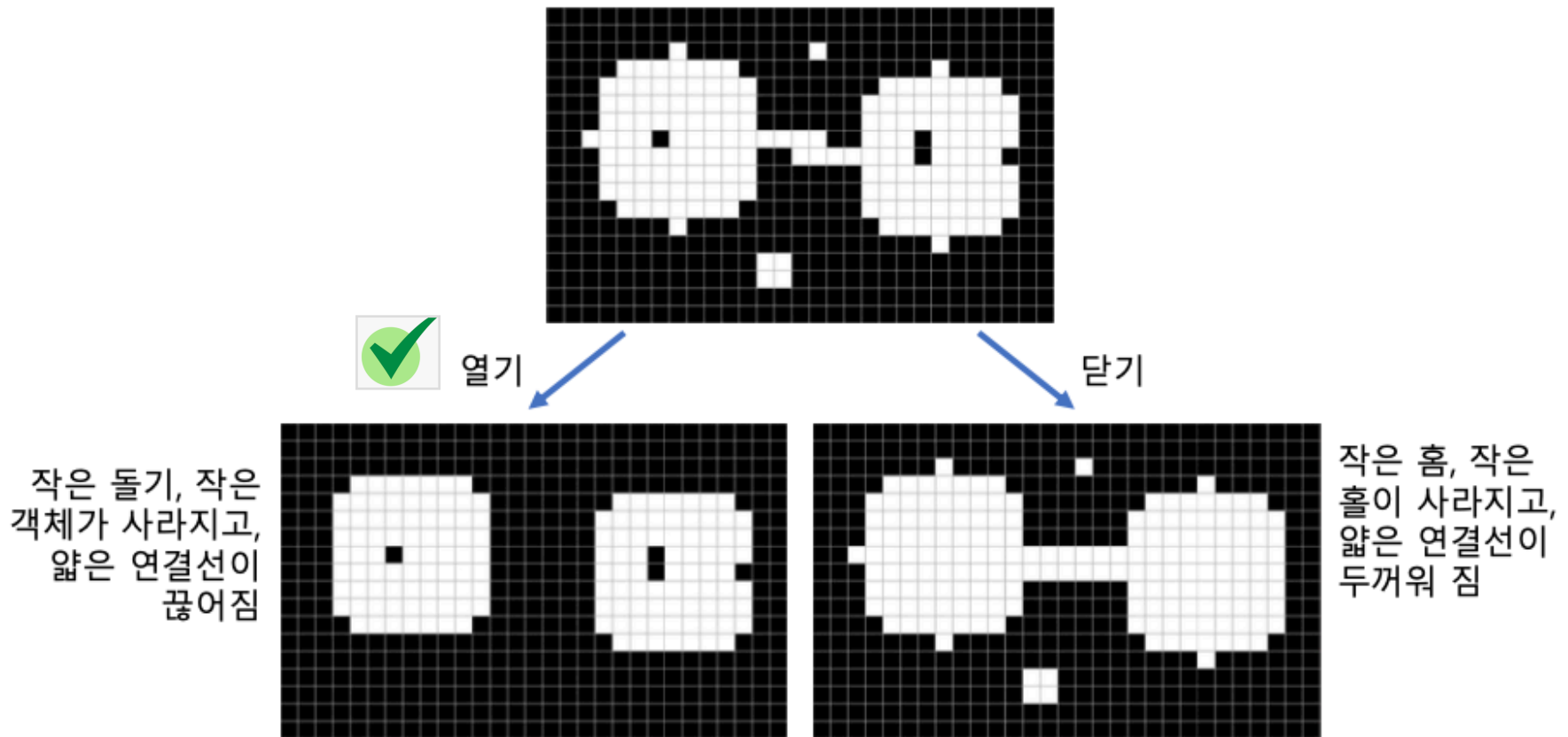
열기 = 침식 → 팽창
Opening = Erosion → Dilation

- 이진 영상의 닫기(closing) 연산

닫기 = 팽창 → 침식
Closing = Dilation → Erosion

모폴로지 (2) : 열기와 닫기

- 열기와 닫기 연산 효과 (3x3 구조 요소 사용)



모폴로지 (2) : 열기와 닫기

■ 범용 모폴로지 연산 함수

```
cv2.morphologyEx(src, op, kernel, dst=None, anchor=None, iterations=None,  
                 borderType=None, borderValue=None) -> dst
```

- src: 입력 영상.
- op: 모폴로지 연산 플래그.

cv2.MORPH_ERODE	침식
cv2.MORPH_DILATE	팽창
cv2.MORPH_OPEN	열기
cv2.MORPH_CLOSE	닫기
cv2.MORPH_GRADIENT	모폴로지 그래디언트 = 팽창 - 침식

- kernel: 커널
- dst: 출력 영상

모폴로지 (2) : 열기와 닫기

■ 열기 연산을 이용한 잡음 제거 예제

```
src = cv2.imread('rice.png', cv2.IMREAD_GRAYSCALE)

dst1 = np.zeros(src.shape, np.uint8)

# src 영상에 지역 이진화 수행 (local_th.py 참고)

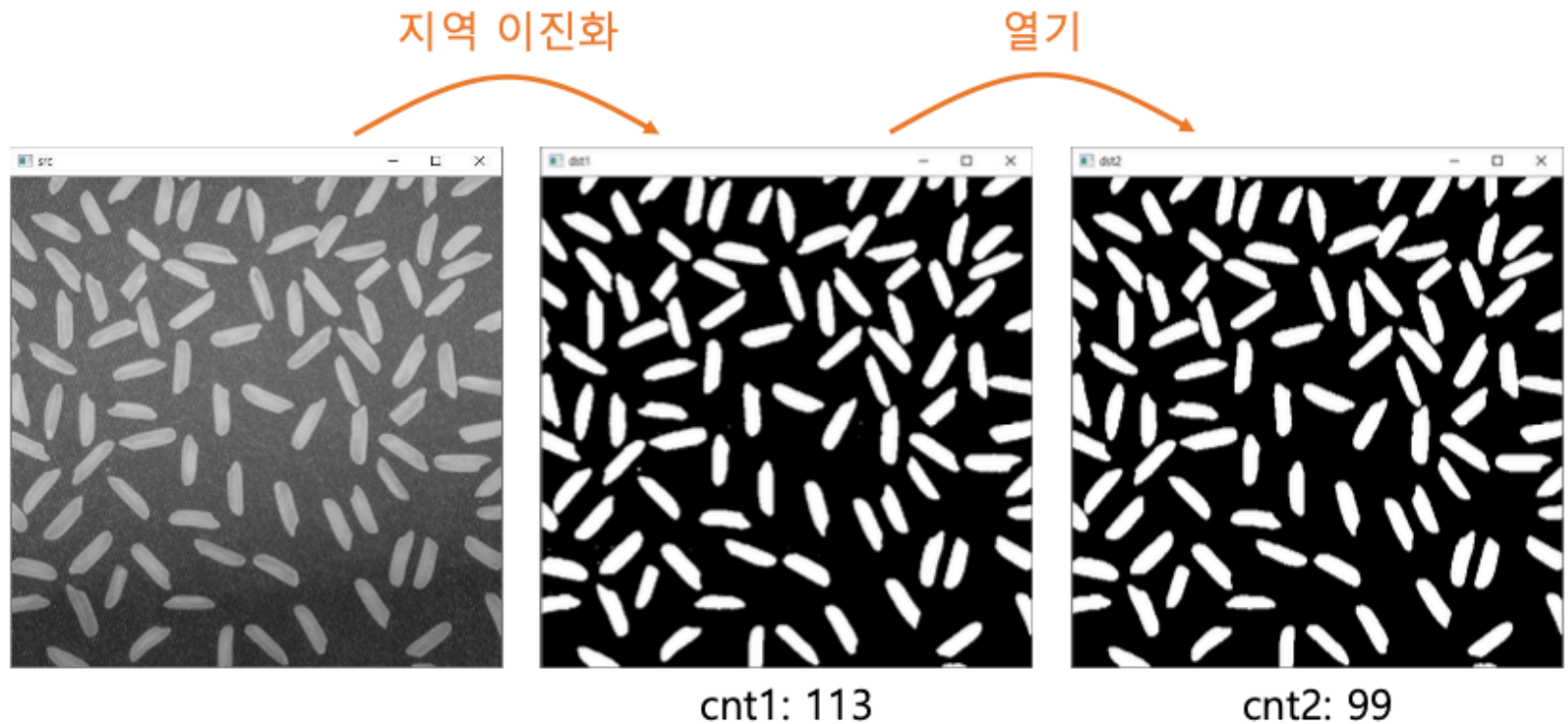
cnt1, _ = cv2.connectedComponents(dst1) # 객체의 개수
print('cnt1:', cnt1)

dst2 = cv2.morphologyEx(dst1, cv2.MORPH_OPEN, None)
#dst2 = cv2.erode(dst1, None)
#dst2 = cv2.dilate(dst2, None)

cnt2, _ = cv2.connectedComponents(dst2)
print('cnt2:', cnt2)
```

모폴로지 (2) : 열기와 닫기

- 열기 연산을 이용한 잡음 제거 예제 실행 결과



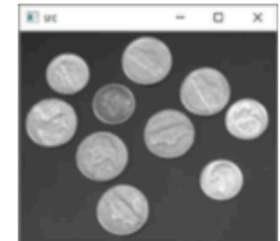
이진 영상 처리

5) 레이블링

객체 단위 분석

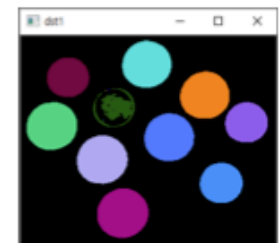
■ 객체 단위 분석

- (흰색) 객체를 분할하여 특징을 분석
- 객체 위치 및 크기 정보, ROI 추출, 모양 분석 등



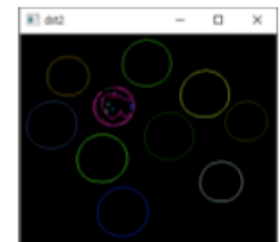
■ 레이블링(Connected Component Labeling)

- 서로 연결되어 있는 객체 픽셀에 고유한 번호를 지정 (레이블맵)
- 영역 기반 모양 분석
- 레이블맵, 바운딩 박스, 픽셀 개수, 무게 중심 좌표를 반환



■ 외곽선 검출(Contour Tracing)

- 각 객체의 외곽선 좌표를 모두 검출
- 외곽선 기반 모양 분석
- 다양한 외곽선 처리 함수에서 활용 가능 (근사화, 컨벡스헐 등)



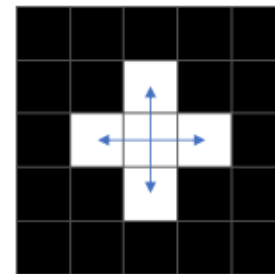
레이블링

■ 레이블링(Labeling)이란?

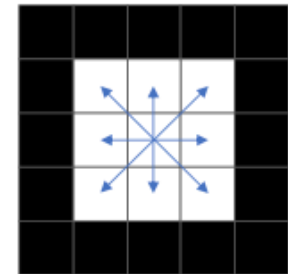
- 동일 객체에 속한 모든 픽셀에 고유한 번호를 매기는 작업
- 일반적으로 이진 영상에서 수행
- OpenCV에는 3.x 버전부터 최신 논문 기반의 레이블링 알고리즘 함수를 제공
- Connected component labeling

■ 픽셀의 연결 관계

- 4-이웃 연결 관계(4-neighbor connectivity)
- 8-이웃 연결 관계(8-neighbor connectivity)



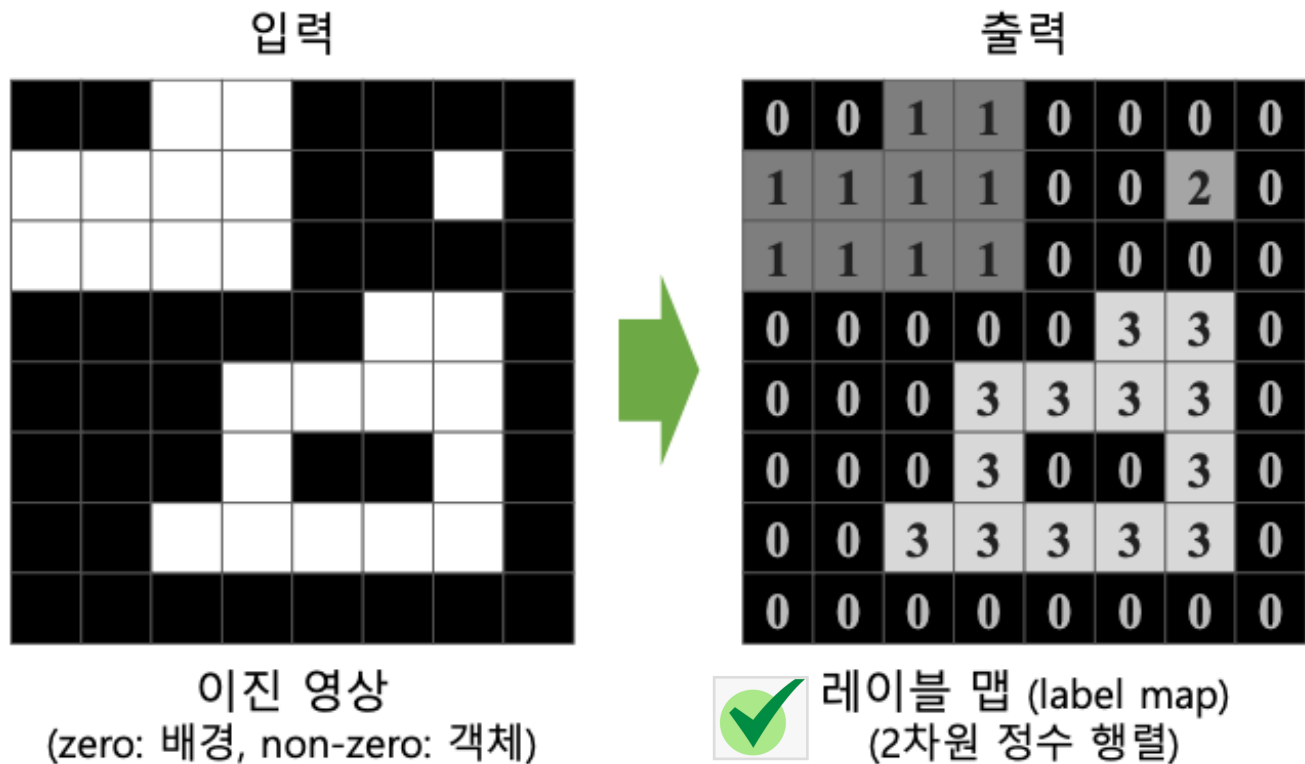
4-이웃 연결 관계



8-이웃 연결 관계

레이블링

레이블링 알고리즘의 입력과 출력



레이블링

■ 레이블링 함수

```
cv2.connectedComponents(image, labels=None, connectivity=None,  
                        ltype=None) -> retval, labels
```

- image: 8비트 1채널 영상
- labels: 레이블 맵 행렬. 입력 영상과 같은 크기. [numpy.ndarray](#).
- connectivity: 4 또는 8. 기본값은 8.
- ltype: labels 타입. cv2.CV_32S 또는 cv2.CV_16S. 기본값은 cv2.CV_32S.
- retval: 객체 개수. N을 반환하면 [0, N-1]의 레이블이 존재 하며, 0은 배경을 의미. (실제 흰색 객체 개수는 N-1개)

레이블링

■ 객체 정보를 함께 반환하는 레이블링 함수



```
cv2.connectedComponentsWithStats(image, labels=None, stats=None,  
                                centroids=None, connectivity=None, ltype=None)  
-> retval, labels, stats, centroids
```

- image: 8비트 1채널 영상
- labels: 레이블 맵 행렬. 입력 영상과 같은 크기. `numpy.ndarray`.
- stats: 각 객체의 바운딩 박스, 픽셀 개수 정보를 담은 행렬.
`numpy.ndarray`. shape=(N, 5), dtype=`numpy.int32`.
- centroids: 각 객체의 무게 중심 위치 정보를 담은 행렬
`numpy.ndarray`. shape=(N, 2), dtype=`numpy.float64`.
- ltype: labels 행렬 타입. `cv2.CV_32S` 또는 `cv2.CV_16S`. 기본값은 `cv2.CV_32S`.

레이블링

■ cv2.connectedComponentsWithStats() 함수 수행 결과의 예

```
retval, labels, stats, centroids = cv2.connectedComponentsWithStats(src)
```

labels

0	0	1	1	0	0	0	0
1	1	1	1	0	0	2	0
1	1	1	1	0	0	0	0
0	0	0	0	0	3	3	0
0	0	0	3	3	3	3	0
0	0	0	3	0	0	3	0
0	0	3	3	3	3	3	0
0	0	0	0	0	0	0	0

retval = 4

8 x 8

stats

0	0	8	8	38	← 배경
0	0	4	3	10	← 1번 객체
6	1	1	1	1	← 2번 객체
2	3	5	4	14	← 3번 객체

면적

바운딩 박스 정보
(x, y, width, height)

centroids

3.615	3.692	← 배경
1.7	1.2	← 1번 객체
6	1	← 2번 객체
4.285	4.785	← 3번 객체

무게 중심의 x 좌표

무게 중심의 y 좌표



레이블링

■ 키보드 영상에서 문자 영역 분할 예제

```
src = cv2.imread('keyboard.bmp', cv2.IMREAD_GRAYSCALE)

_, src_bin = cv2.threshold(src, 0, 255, cv2.THRESH_OTSU)
cnt, labels, stats, centroids = cv2.connectedComponentsWithStats(src_bin)

dst = cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)

for i in range(1, cnt):
    (x, y, w, h, area) = stats[i]

    if area < 20:
        continue

    cv2.rectangle(dst, (x, y, w, h), (0, 255, 255))
```

레이블링

- 키보드 영상에서 문자 영역 분할 예제 실행 결과

